

SoftBoost

By: Ava Kirkland

Introduction

The project started with the base model of AdaBoost. The extension I implemented was to mesh AdaBoost and SVM loss functionality inspired by Rätsch's *Soft Margins for AdaBoost*, with quadratic loss with slack variables similar to support vector machines (2001). As he mentions, AdaBoost uses a hard margin pattern for predicting samples. AdaBoost does not perform well when there is noise in the data or overlapping class distributions because it chases outliers and overfits to them (2001). The goal is to improve the Adaboost model's robustness when it interacts with overlapping class distributions and noise. Therefore, replace hard margins with soft margins to allow misclassifications in the margins and to reduce overfitting to abnormal data points.

Dataset Description

The dataset used was the Titanic Dataset on Kaggle^[2]. The dataset has 891 samples (people) out of the 2224 people on the Titanic in 1912 when the ship sank. The target feature to predict is whether the individual survived the Titanic. The other features provided include passenger ID, ticket class (1st, 2nd, 3rd class), name, gender, age, number of siblings/spouses on board, number of parents/children on the Titanic, ticket number, passenger fare, cabin number, and embarked (what port they entered the Titanic on).

Some of these features are not useful in prediction, and some have some missing values. So data cleaning needs to occur after loading the dataset and before giving it to the models. I looked at what others have done with this dataset to prepare it for prediction models. A Python notebook by babyxkush, helped with cleaning valuable features with missing data for some people: age and embark^[3]. I removed the Cabin column, 77% missing values. Removed other columns with unique values for each person: passenger ID, name, and ticket number. Embarked had 2 missing values. Over 70% of the people had an embarked value of 'S' = Southampton. I set the 2 missing elements to be S. Age was missing in 20% of samples. However, age is very important in determining if someone survived the Titanic. Solution: Find samples with the same gender and ticket class and give the missing sample the median age of the group.

Further cleaning of the data included one-hot-encoding the embarked and gender columns so the model can more easily interpret patterns^[5]. Last but not least was changing the y (target) numpy array from $\{0,1\}$ to $\{-1,1\}$ to conform to AdaBoost standards. The Titanic dataset was selected because the target feature is not 50% yes and no, and has an overlapping class distribution. This can be difficult for the vanilla AdaBoost model.

Methodology

Conceptually, I combined the loss functionality of AdaBoost and SVM. That alters how the losses, new weights, and predictions are calculated. Articles: *Regularizing AdaBoost* by Riitsch and *Soft Margins for AdaBoost* by Rätsch, the loss matrix returns -1 if the prediction was incorrect and 1 otherwise, for the hinge loss weights (1998)(2001)^{[1][4]}. I changed it to a loss of 0 if the sample was correctly classified, so it has no impact. Then return 1 if the sample is in the

margin or misclassified for the hinge loss weights, and has an impact. The articles' equation for quadratic AdaBoost with SVM slack variables: minimize

$$\|b\|^2 + C \sum_i \xi_i$$

b = weight, C is a constant, ξ_i is the slack variable at that sample, L is the loss

Both Articles use constraints of $\sum_{t=1}^T b_t L_{i,t} \geq 1 - \xi_i$ with the constraints of

$$b_t \geq 0, \xi_i \geq 0,^{[1][4]}$$

Big picture in code, the mesh model called SoftBooster with an AdaBoost model base, and the update steps are a mix of AdaBoost and SVM functionality. In the update step of the model, it calculates the loss by adding the exponential loss and hinge loss together. Then the new weights are a combination of expo and hinge weights. The predictions are based on an ensemble score used in SVM. The sign selects the class to predict, and the magnitude of the number gives the confidence that that prediction is correct.

In the fit function, it updates the exponential, hinge, and combinational loss and records them during the iterations of developing decision stumps. The combined loss function is the exponential loss from vanilla AdaBoost, added to the hinge loss from SVM loss functionality.

$$L(m) = lam * e^{-m} + (1 - lam) * \max(0, 1 - m)$$

m = margin for a sample, Exponential loss = e^{-m} , Hinge Loss = $\max(0, 1 - m)$, and the lam = constant variable that controls the ratio of expo and hinge loss in the combinational loss. If the $lam = 0$, only hinge loss will be used. If $lam = 1.0$, only the expo loss will be used. If $0 < lam < 1.0$, then the combinational loss is impacted by both expo and hinge losses.

The new weights are evaluated by the combined weights:

$$weights = lam * e^{-y_i * F_i} + (1 - lam) * [y_i * F_i < 1]$$

y_i = is the true value of the target, F_i = the ensemble score, exponential weight = $e^{-y_i * F_i}$,

and hinge weight = $[y_i * F_i < 1]$ is 1 if true and 0 otherwise.

The prediction is based on the ensemble score: $F = \sum_t alpha_t * h_t$.

$alpha$ = a model's weight with one per stump, h = predicted value of the stump.

The sign on F gives what class to predict, and the magnitude of the value states the confidence of this prediction.

Results

Results of Vanilla AdaBoost with 50 decision stumps:

Round 10: Error: 0.4631, Alpha = 0.0738

Round 20: Error: 0.4747, Alpha = 0.0507

Round 30: Error: 0.4838, Alpha = 0.0324

Round 40: Error: 0.4857, Alpha = 0.0286

Round 50: Error: 0.4900, Alpha = 0.0201

Vanilla AdaBoost Results

Training Accuracy: 0.8216

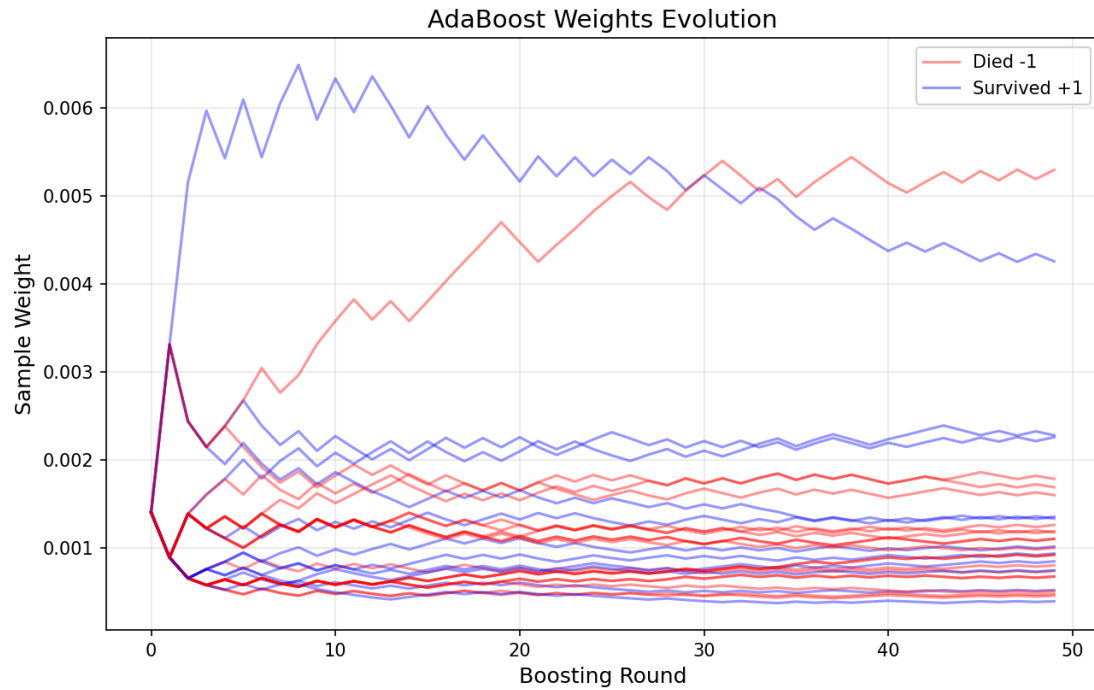
Test Results

Accuracy: 0.8212

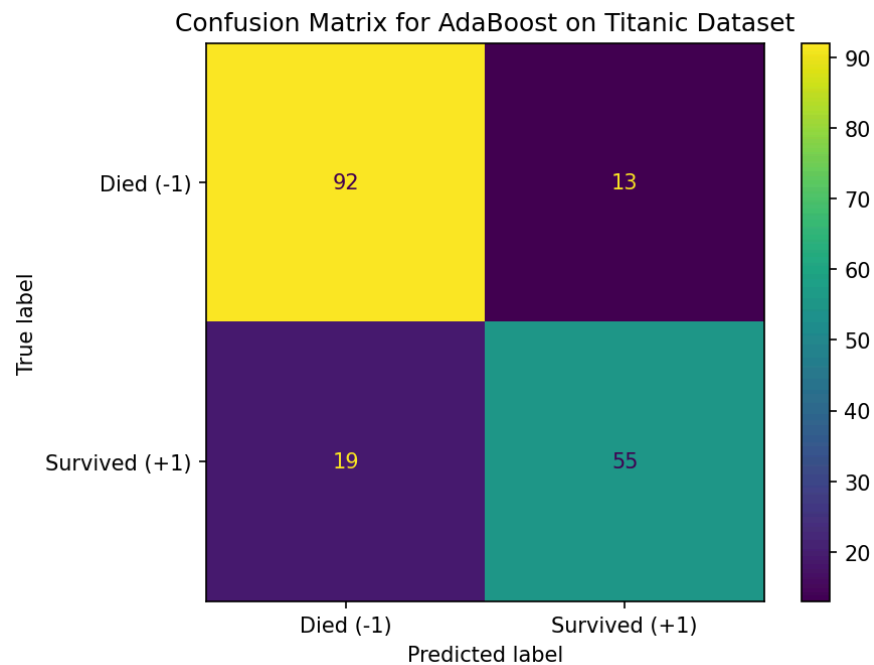
Precision: 0.8088

Recall: 0.7432

F1 Score: 0.7746



A plot that illustrates the first 30 test values weights evolution over the 50 Boosting Rounds.



A confusion matrix of the AdaBoost Testing data. Titanic Survival Prediction notebook by kimobadr showed how to use Sklearn for a confusion matrix ^[6].

Below are results for the SoftBooster model with 50 decision stumps and various lam values:

SoftBoost (lam = 0.0)

Round 10: Epsilon: 0.0000, Alpha: 11.5129

Round 20: Epsilon: 0.0000, Alpha: 11.5129

Round 30: Epsilon: 0.0000, Alpha: 11.5129

Round 40: Epsilon: 0.0000, Alpha: 11.5129

Round 50: Epsilon: 0.0000, Alpha: 11.5129

Soft AdaBoost Results (lam=0.0)

Training Accuracy: 0.7879

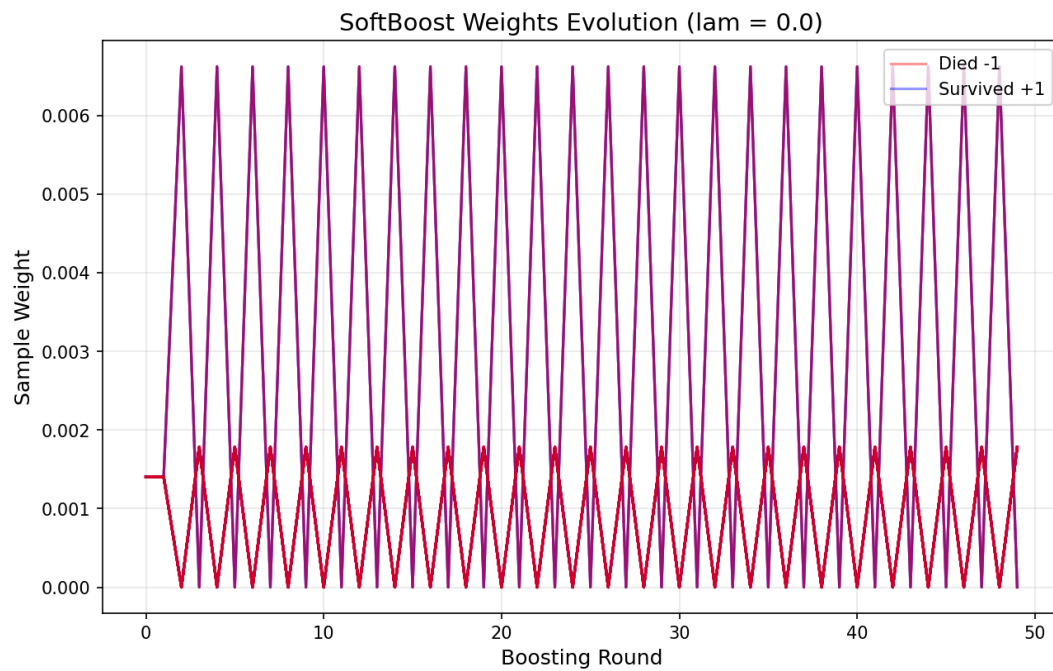
Test Results

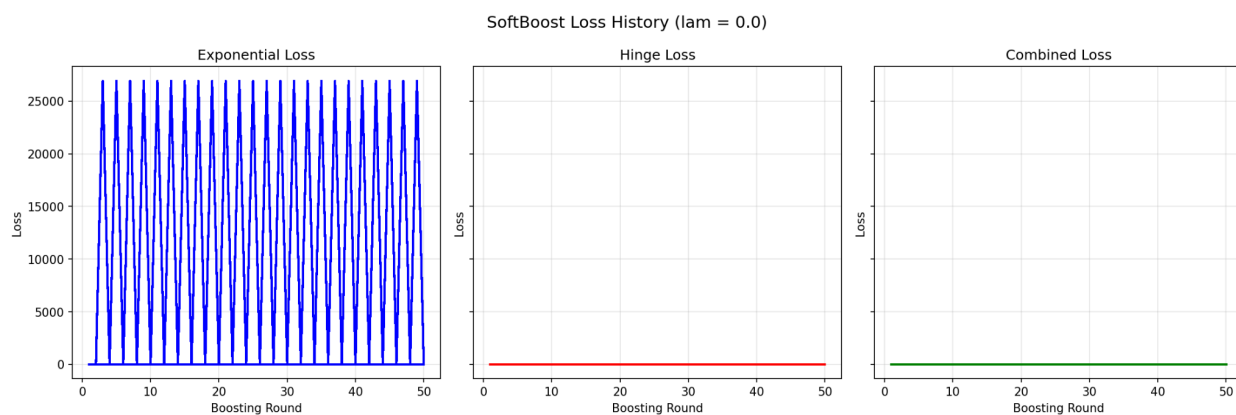
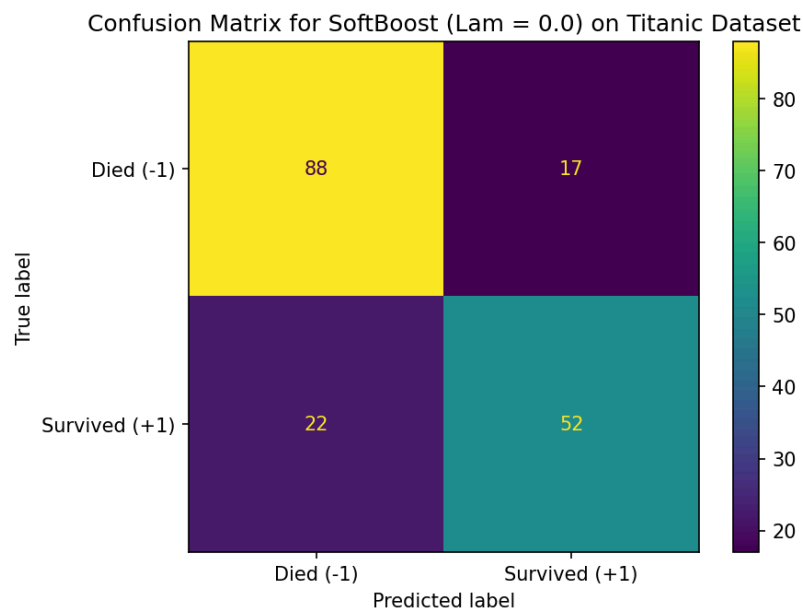
Accuracy: 0.7821

Precision: 0.7536

Recall: 0.7027

F1 Score: 0.7273





Plot of the Exponential (AdaBoost), Hinge (SVM), and combined (SoftBoost) loss over the 50 rounds. It plots the first 50 values of each loss function over the iterations.

SoftBoost (lam = 0.1)

Round 10: Epsilon: 0.2128, Alpha: 0.6540
 Round 20: Epsilon: 0.2369, Alpha: 0.5848
 Round 30: Epsilon: 0.0775, Alpha: 1.2387
 Round 40: Epsilon: 0.2128, Alpha: 0.6539
 Round 50: Epsilon: 0.2369, Alpha: 0.5848

Soft AdaBoost Results (lam=0.1)

Training Accuracy: 0.7879

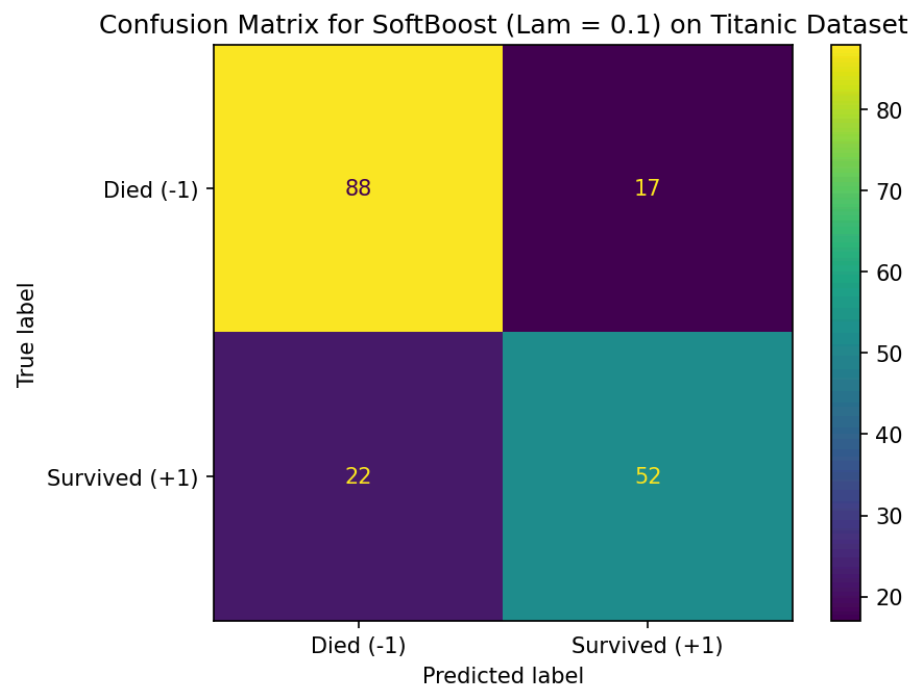
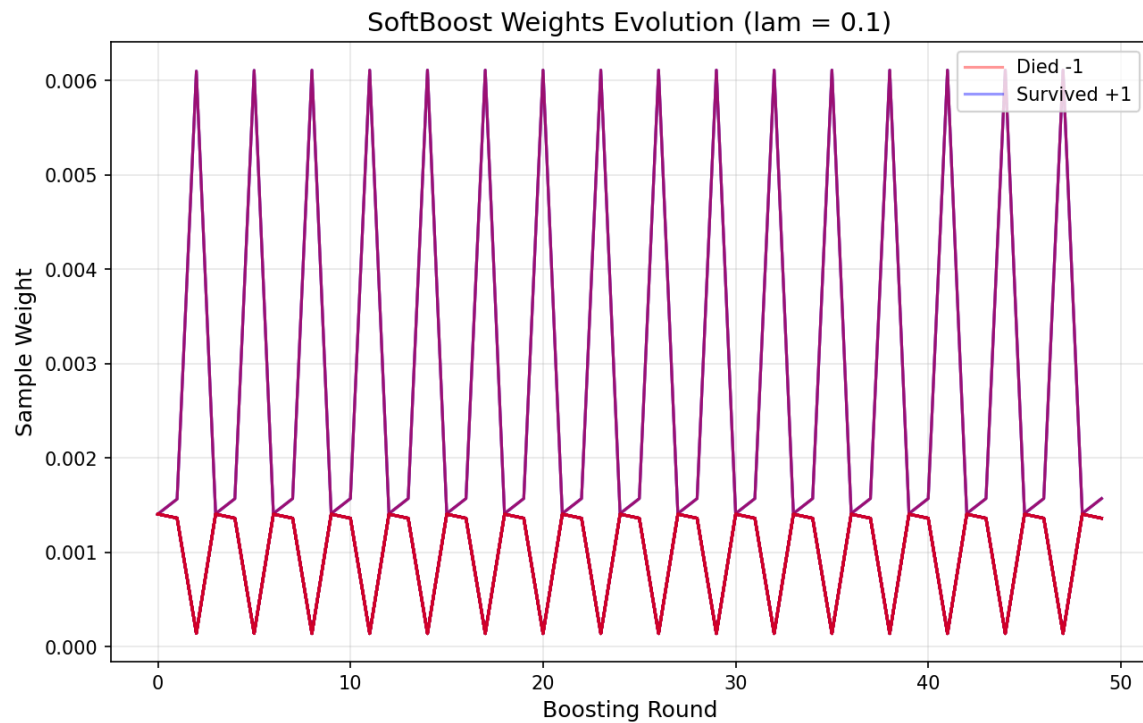
Test Results

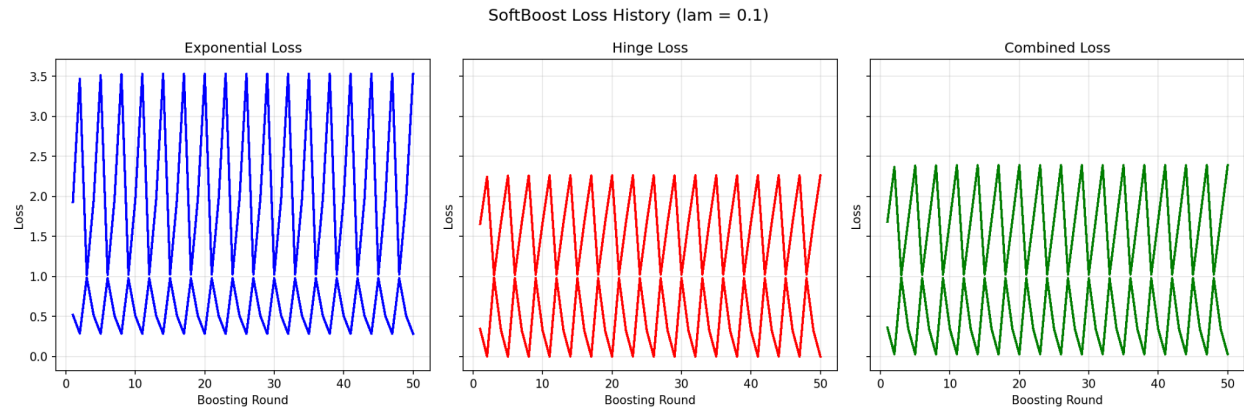
Accuracy: 0.7821

Precision: 0.7536

Recall: 0.7027

F1 Score: 0.7273





SoftBoost ($\text{lam} = 0.5$)

Round 10: Epsilon: 0.4438, Alpha: 0.1128

Round 20: Epsilon: 0.4435, Alpha: 0.1135

Round 30: Epsilon: 0.4633, Alpha: 0.0736

Round 40: Epsilon: 0.4851, Alpha: 0.0298

Round 50: Epsilon: 0.4757, Alpha: 0.0487

Soft AdaBoost Results ($\text{lam}=0.5$)

Training Accuracy: 0.8244

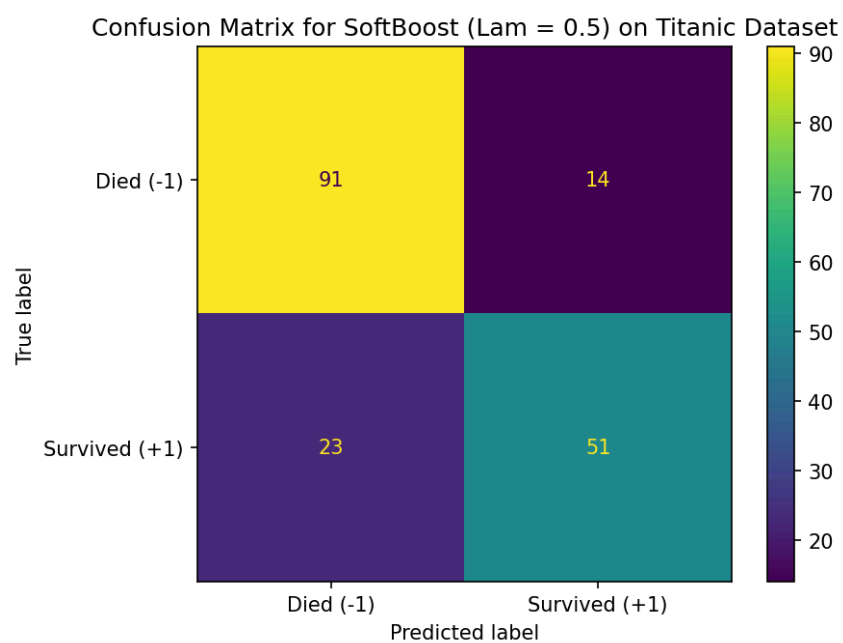
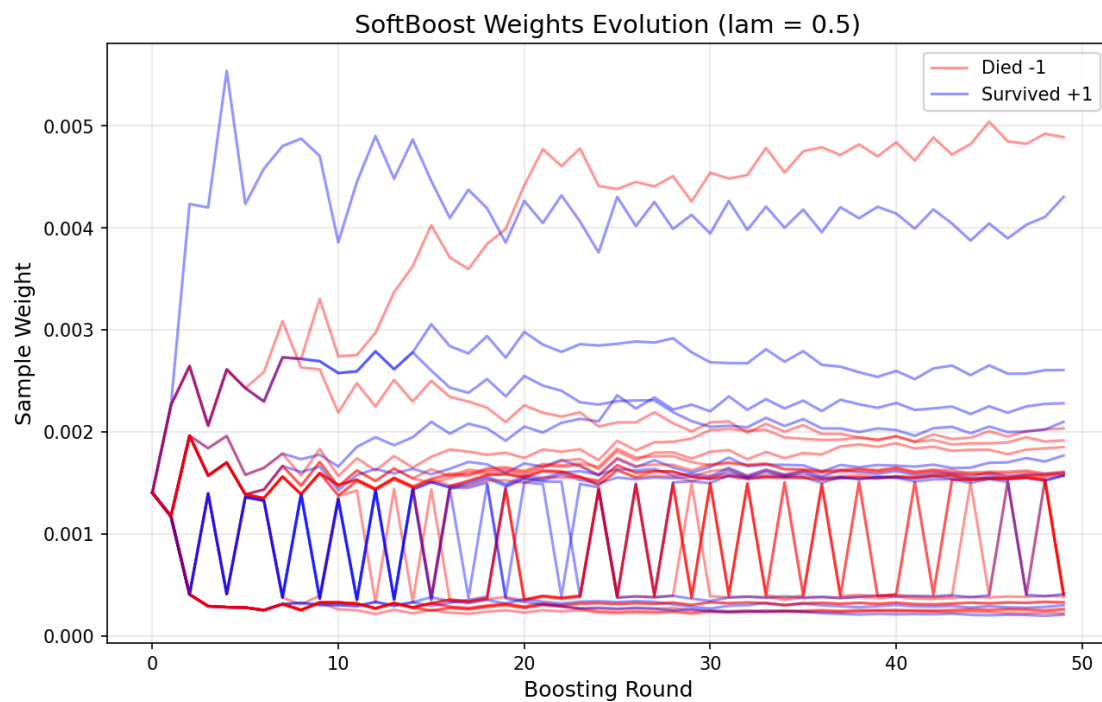
Test Results

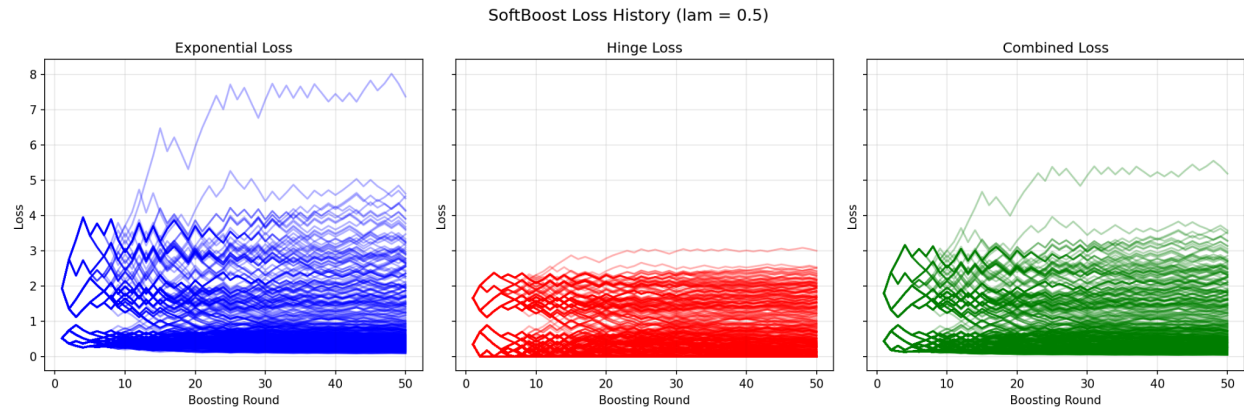
Accuracy: 0.7933

Precision: 0.7846

Recall: 0.6892

F1 Score: 0.7338





SoftBoost (lam = 0.9)

Round 10: Epsilon: 0.4561, Alpha: 0.0880

Round 20: Epsilon: 0.4771, Alpha: 0.0459

Round 30: Epsilon: 0.4783, Alpha: 0.0434

Round 40: Epsilon: 0.4849, Alpha: 0.0302

Round 50: Epsilon: 0.4893, Alpha: 0.0213

Soft AdaBoost Results (lam=0.9)

Training Accuracy: 0.8202

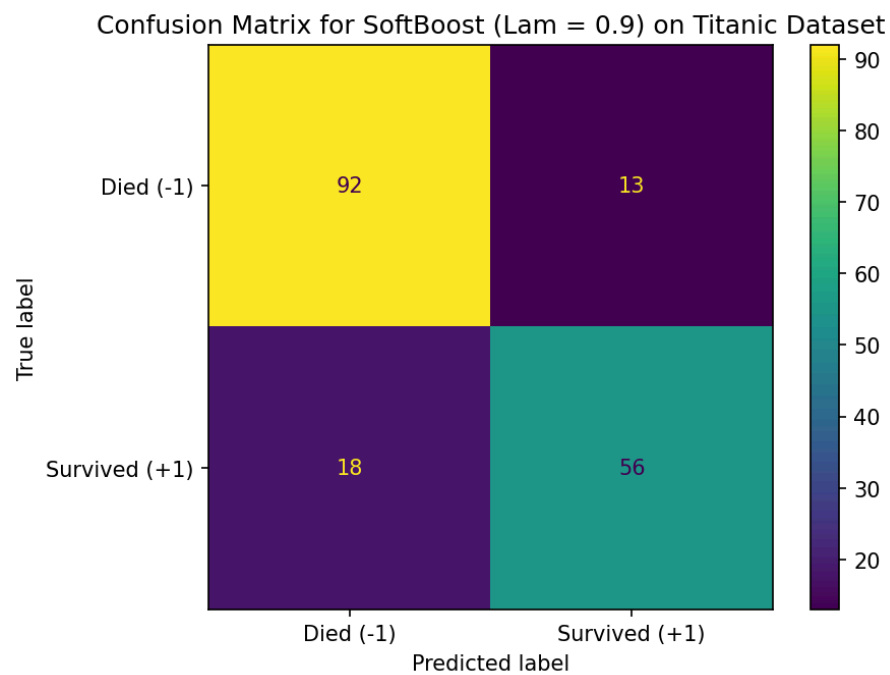
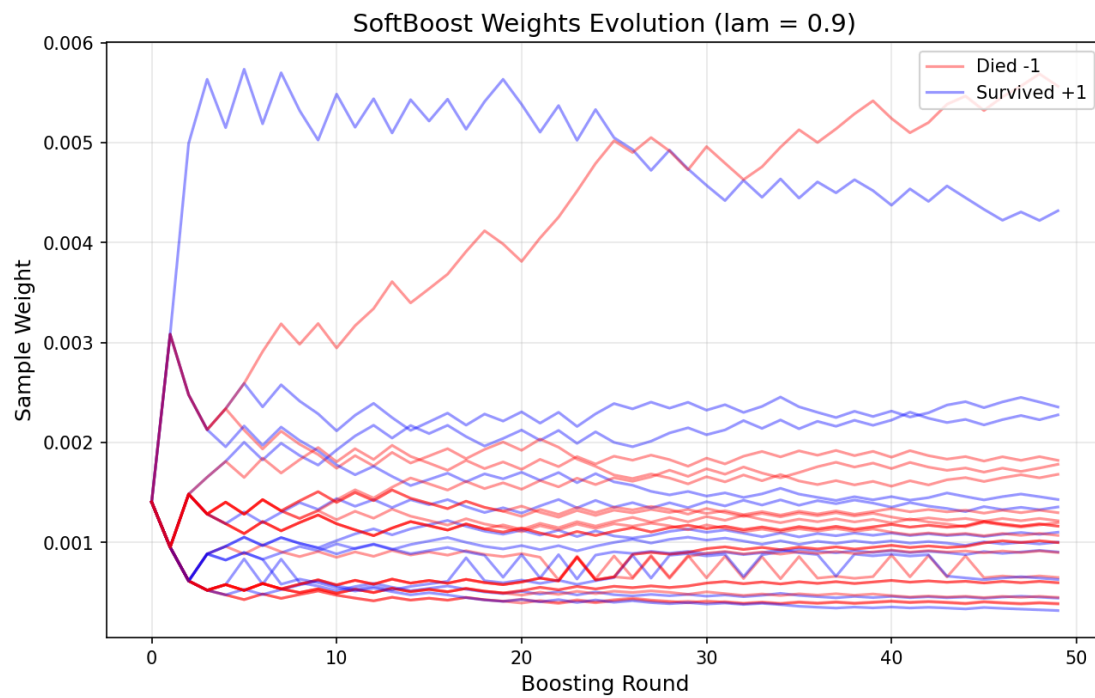
Test Results

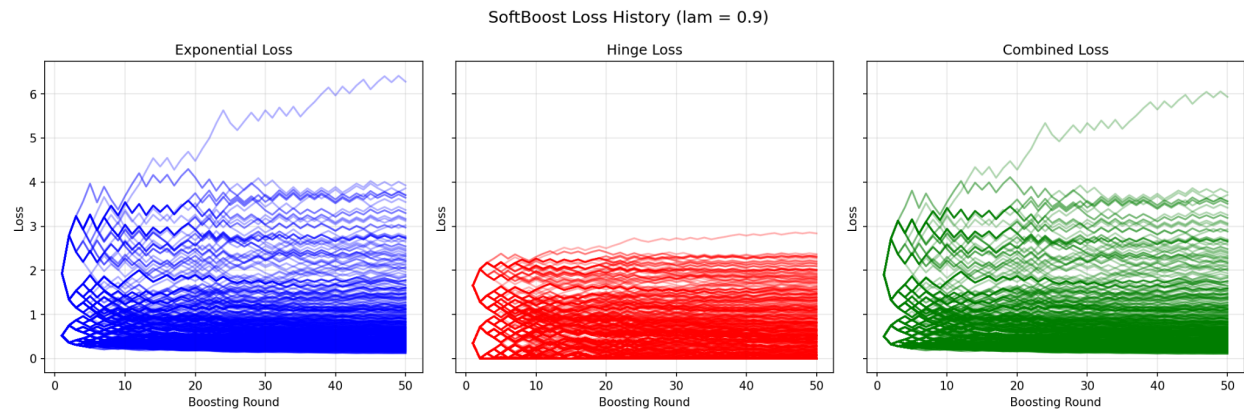
Accuracy: 0.8268

Precision: 0.8116

Recall: 0.7568

F1 Score: 0.7832





SoftBoost (lam = 1.0)

Round 10: Epsilon: 0.4631, Alpha: 0.0738

Round 20: Epsilon: 0.4747, Alpha: 0.0507

Round 30: Epsilon: 0.4838, Alpha: 0.0324

Round 40: Epsilon: 0.4857, Alpha: 0.0286

Round 50: Epsilon: 0.4900, Alpha: 0.0201

Soft AdaBoost Results (lam=1.0)

Training Accuracy: 0.8216

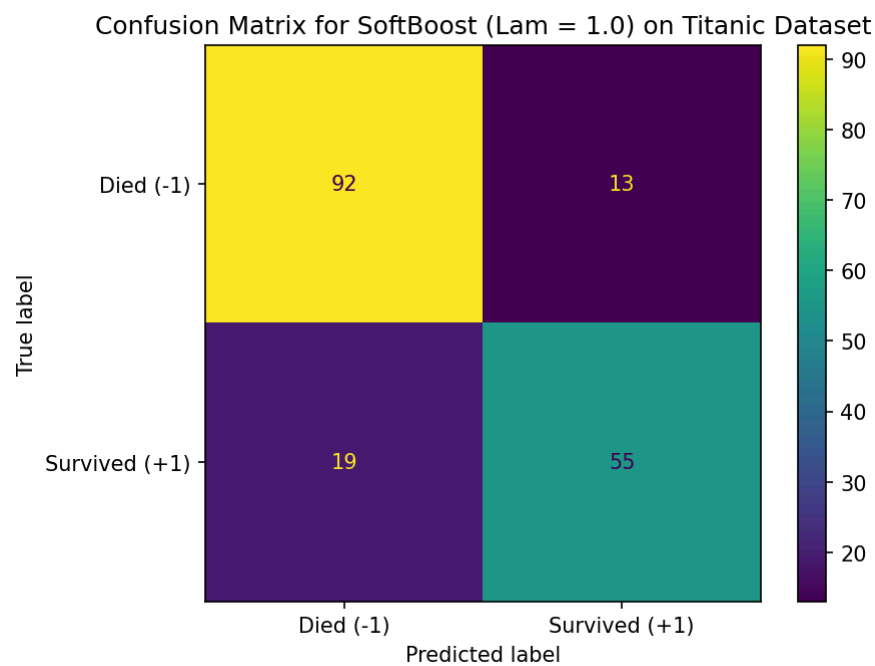
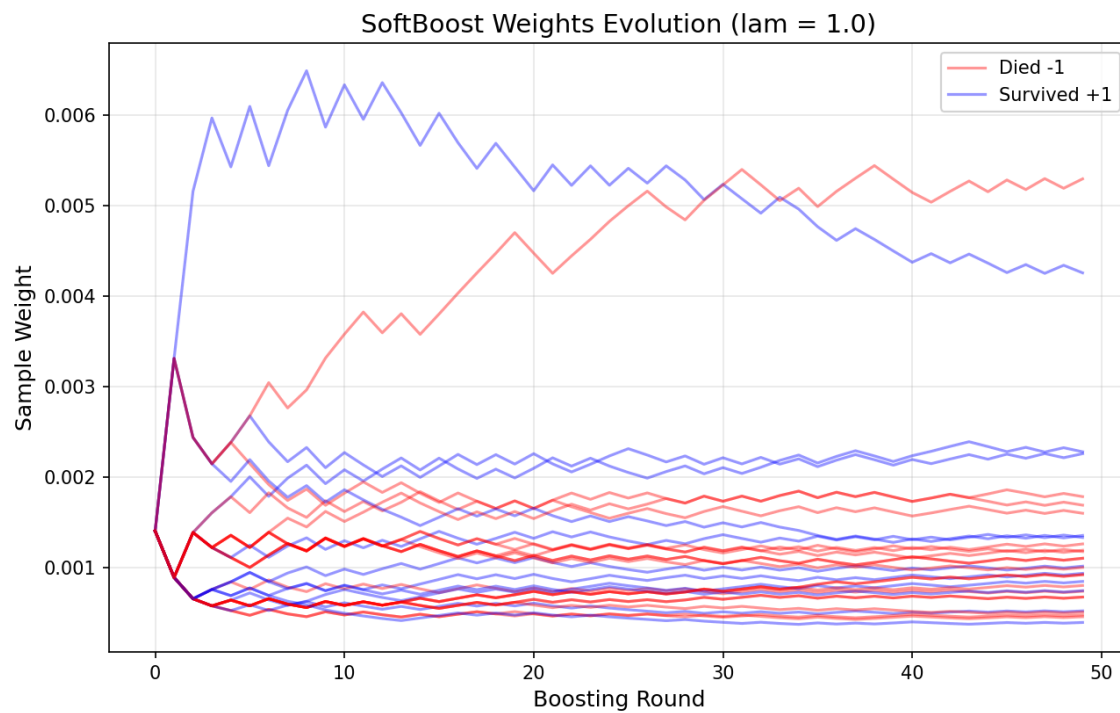
Test Results

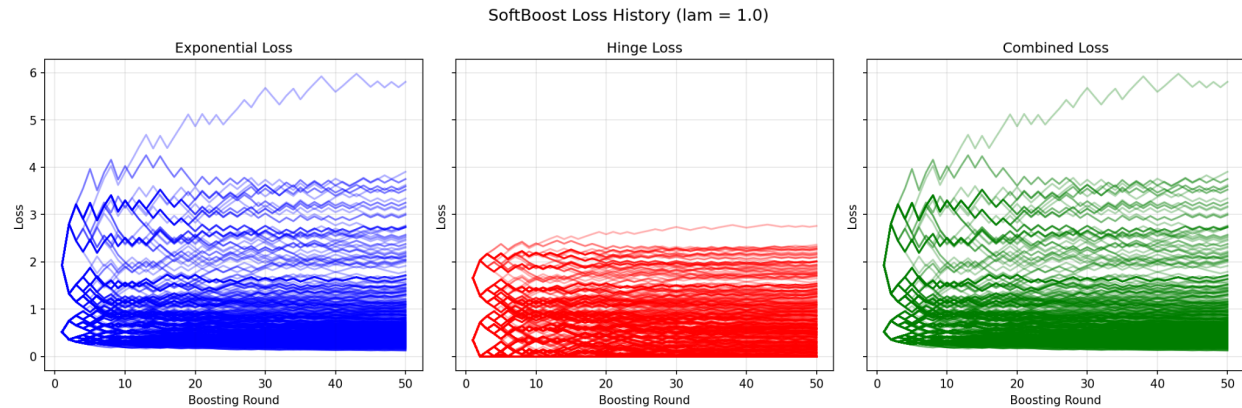
Accuracy: 0.8212

Precision: 0.8088

Recall: 0.7432

F1 Score: 0.7746





Discussion

The AdaBoost weight evolution plot showed that the graph was overfitting to overlapping class samples, with a few samples having high weights while the rest had very little weight and flat lines.

The worst SoftBoost was $\text{lam} = 0.0$ and 0.1 . They had different epsilons and alphas, but the results measured were all the same. The training accuracy was 78.79%, and the testing accuracy of 78.21%. This is when only hinge loss is utilized or exponential loss has very little impact. It makes sense that a (mostly) pure SVM loss functionality doesn't work well with an AdaBoost base model. The weights over time for $\text{lam} = 0.0$ exploded in magnitude. Both have a sawtooth graph indicating possible underfitting.

The best SoftBoost was $\text{lam} = 0.9$. Training accuracy of 82.02%. Testing accuracy of 82.68%. Precision values of 81.16%. Recall is 75.68%. F1 score is 78.32%. $\text{Lam} = 0.9$ beat regular AdaBoost on every measurement for the tested data. The weights evolution graph shows an increase of samples with higher weights in later boosting rounds. But it still chases some overlapping class samples. At $\text{lam} = 0.9$, the model is mostly AdaBoost with a little hinge influence, to allow a few misclassifiers in the margin and soft margins.

At $\lambda = 1.0$, the model is a vanilla AdaBoost model, verified by having the same values and graphs as the regular AdaBoost model.

At $\lambda = 0.5$, the model is half AdaBoost and half SVM loss function. It had the highest training accuracy of 82.44%, but about -3% on testing accuracy with 79.33%. The weight's evolution showed more samples had an impact on the next decision stump than other models.

Overall, the SoftBooster helped with generalization, when it uses mostly an AdaBoost loss with slight influence from SVM to allow softer margins. Comparing $\lambda = 0.9$ to AdaBoost, there an increase in testing accuracy 0.56%, precision 0.28%, recall 1.26%, and F1 score 0.86%.

I was surprised that $\lambda = 0.5$ didn't do as well or better than the regular AdaBoost. Also, I was shocked that recall, precision, and F1 score were in the same range of high 60s to low 80s for all versions of the model. Usually they were in the low to mid-70s percentage.

If I had more time, I would explore implementing line search for the alpha value and use name prefixes to aid in assigning missing age values. I would consider clipping the weight value to remove the explosion at $\lambda = 0.0$, and test different numbers of boosting rounds.

Reflection

I learned about the importance of cleaning the data so that the model can make useful predictions and interpret them. A number can only go so far. Sometimes a picture or graph can express more emphasis than a single number can. So how you show your data is important. Finding ways to graph your data can be difficult, but it is valuable. I learned more about the update step in machine learning models and how different loss implementations can alter how well or poorly a model performs.

Citations

1. Rätsch, G., Onoda, T., & Müller, K.-R. . (2001). Soft Margins for AdaBoost. *Machine Learning*, 42(3), 287–320. <https://doi.org/10.1023/a:1007618119488>
2. Titanic Dataset. (n.d.). Wwww.kaggle.com.
<https://www.kaggle.com/datasets/yasserh/titanic-dataset>
3. babyxkush. (2026, February 25). *Titanic Prediction*. Kaggle.com; Kaggle.
<https://www.kaggle.com/code/babyxkush/titanic-prediction>
4. Riitsch, G., Onoda, T., & Iiller, K. (n.d.). *Regularizing AdaBoost*. Retrieved May 3, 2026, from
https://proceedings.neurips.cc/paper_files/paper/1998/file/71a58e8cb75904f24cde464161c3e766-Paper.pdf
5. *pandas.get_dummies* — *pandas 1.2.4 documentation*. (n.d.). Pandas.pydata.org.
https://pandas.pydata.org/docs/reference/api/pandas.get_dummies.html
6. kimobadr. (2026, April 3). *Titanic Survival Prediction*. Kaggle.com; Kaggle.
<https://www.kaggle.com/code/kimobadr/titanic-survival-prediction>

NumPy Developers. (n.d.). *numpy.array* — *NumPy v1.19 Manual*. Numpy.org.

<https://numpy.org/doc/stable/reference/generated/numpy.array.html>

Below are the AI citations for debugging sections:

Google. (2026,4,18). *Pandas DataFrame Indexing Error Fix* [Generative AI chat]

Google Gemini. [Pandas DataFrame Indexing Error Fix - Google Gemini](#)

Microsoft. (2026, 4,23) Debugging Decision Stump Threshold Error [Generative AI chat]

Copilot, from <https://copilot.microsoft.com>

Anthropic. (2026). *Claude* (claude-sonnet-4-6) [Large language model]. <https://claude.ai>